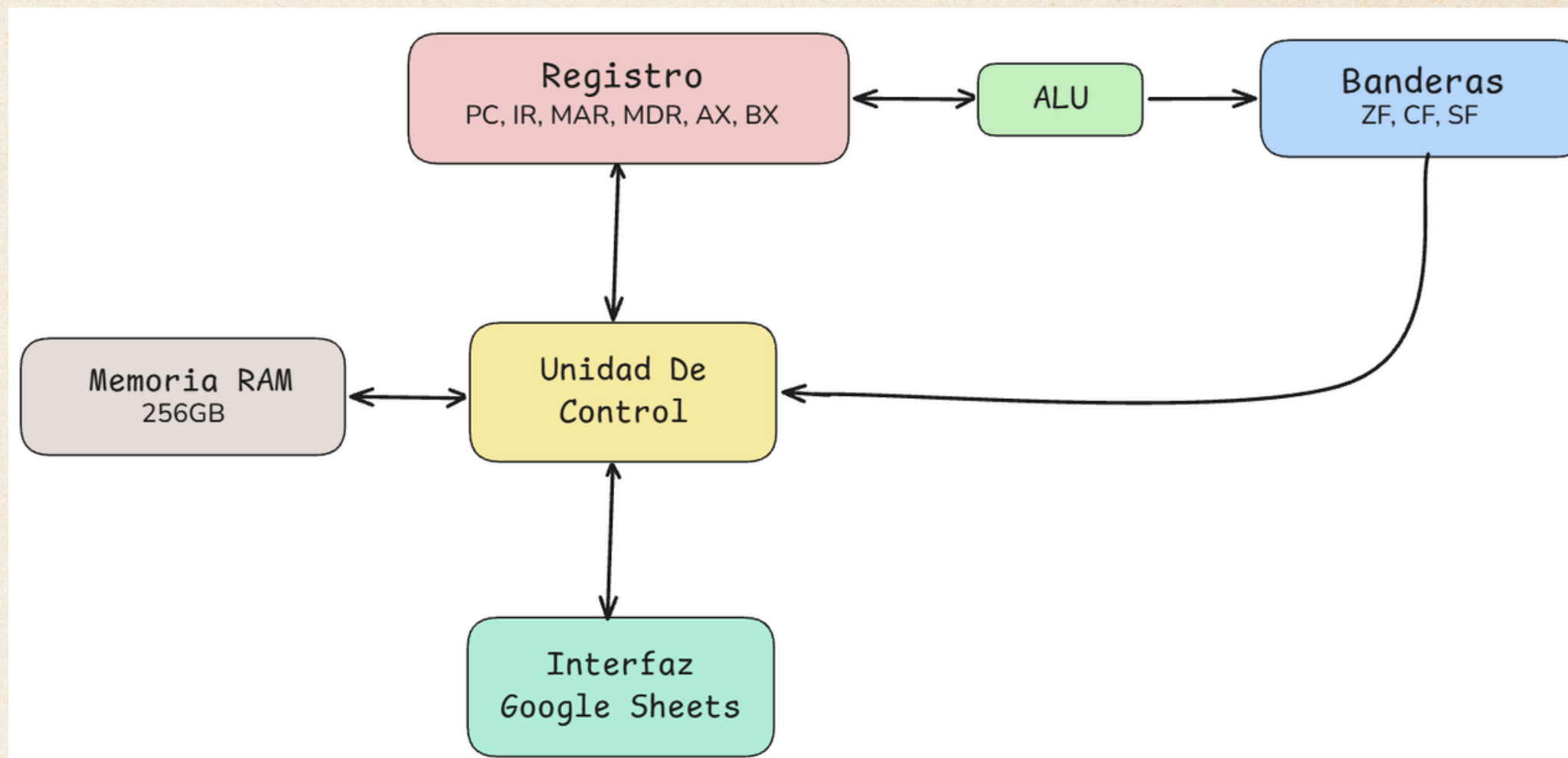


PRESENTACIÓN DE

SIMULADOR DE PC DE 8 BITS

Diego Andrade Canedo

Arquitectura del Sistema



- La Unidad de Control coordina el ciclo de instrucción.
- La RAM almacena instrucciones y datos.
- Los registros mantienen el estado interno de la CPU.
- La ALU realiza las operaciones aritméticas y lógicas.
- Las banderas reflejan el resultado de las operaciones.
- La interfaz en Google Sheets permite visualizar y controlar la ejecución.

Estructura Google Sheet

Hoja CPU

REGISTROS DEL CPU		Banderas	
PC	00	ZF	0
IR	00	CF	0
MAR	00	SF	0
MDR	00		
AX	00		
BX	00		

STEP

RUN

PAUSE

RESET

LOAD PROGRAM

Estado del CPU	
Fase Actual	FETCH
Estado	READY
Instruccion	
Ultima Operacion	

Velocidad (ms)	100
Máx. instrucciones	500

LOG DE MICRO-OPERACIONES		
Hora	Fase	Operacion

Hoja RAM

1	Direccion	Hexadecimal	Binario	Tipo
2	00h	10	00010000	CODE
3	01h	00	00000000	CODE
4	02h	11	00010001	CODE
5	03h	05	00000101	CODE
6	04h	50	01010000	CODE
7	05h	42	01000010	CODE
8	06h	62	01100010	CODE
9	07h	04	00000100	CODE
10	08h	22	00100010	CODE
11	09h	80	10000000	CODE
12	0Ah	00	00000000	CODE
13	0Bh	00	00000000	CODE
14	0Ch	00	00000000	CODE
15	0Dh	00	00000000	CODE
16	0Eh	00	00000000	CODE
17	0Fh	00	00000000	CODE

	A	B	C	D
235	E9h	00	00000000	DATA
236	EAh	00	00000000	DATA
237	EBh	00	00000000	DATA
238	ECh	00	00000000	DATA
239	EDh	00	00000000	DATA
240	EEh	00	00000000	DATA
241	EFh	00	00000000	DATA
242	F0h	00	00000000	DATA
243	F1h	00	00000000	DATA
244	F2h	00	00000000	DATA
245	F3h	00	00000000	DATA
246	F4h	00	00000000	DATA
247	F5h	00	00000000	DATA
248	F6h	00	00000000	DATA
249	F7h	00	00000000	DATA
250	F8h	00	00000000	DATA
251	F9h	00	00000000	DATA
252	FAh	00	00000000	DATA
253	FBh	00	00000000	DATA
254	FCh	00	00000000	DATA
255	FDh	00	00000000	DATA
256	FEh	00	00000000	DATA
257	FFh	00	00000000	DATA

Hoja Programa

	A	B	C
1	Programa	Dirección	Bytes
2	MOV AX, 0	00H	10 00
3	MOV BX, 5	02H	11 05
4	INC AX	04H	50
5	CMP AX, BX	05H	42
6	JNZ 04H	06H	62 04
7	STORE [80H], A	08H	22 80
8	HLT	0AH	0

Desarrollo y organización en GitHub

Organizacion

Simulador-CPU-8-Bits

Public

Unpin

main

1 Branch

0 Tags

Go to file

Add file

<> Code

Dieg0-andrade

refactor: actualizar pruebas de entrada al simulador final

0ad1e92 · 2 hours ago

47 Commits

docs

docs: definir arquitectura del simulador

2 days ago

src

refactor: actualizar pruebas de entrada al simulador final

2 hours ago

README.md

docs: actualizar README con arquitectura e ISA final

2 hours ago

Simulador-CPU-8-Bits / src

Dieg0-andrade refactor: actualizar pruebas de entrada al simulador final

Name

..

ALU.gs

Assembler.gs

CPU.gs

ControlUnit.gs

ISA.gs

Interface.gs

Main.gs

Memory.gs

Tests.gs

47 Commits

Closed 18

Tablero Kanban

Proyecto Simulador de CPU de 8 bits

View 1 + New view

Filter by keyword or by field

Backlogs

To Do

In Progress

Testing

Done

1. Configurar estructura inicial del proyecto

2. Diseñar la arquitectura general del simulador

3. Diseñar memoria RAM de 256 bytes

4. Implementar operaciones de lectura y escritura de memoria

5. Implementar registros del CPU

6. Implementar banderas de estado del CPU

7. Implementar operaciones aritméticas de la ALU

8. Implementar operaciones lógicas de la ALU

9. Implementar decodificador de instrucciones

10. Implementar fase Fetch

11. Implementar fase Decode

12. Implementar fases Execute y Stop

Programa de demostración

Codigo

```
MOV AX, 0
MOV BX, 5
INC AX
CMP AX, BX
JNZ 04H
STORE [80H], AX
HLT
```

- AX empieza en 0.
- BX contiene 5.
- AX aumenta de uno en uno.
- CMP compara AX con BX.
- JNZ repite el ciclo mientras sean distintos.
- Cuando AX llega a 5, se guarda el valor en RAM[80H].
- HLT detiene la CPU.

Resultado

```
AX = 05H
BX = 05H
ZF = 1
RAM[80H] = 05H
Estado = HALTED
```